Introduction to Python Data Types

Week 7 Assignment: Summarize the Python Programming Concepts

Name: Jiya Jha

University: Westcliff University

PRG 300: Python Programming

Prof. Mr. Tek Raj Pant

Date: 26 April, 2026

Part 1: Basic Python Syntax

Python has easy and comprehensible syntax with the indented code blocks defined by indentation. It can handle various types of data including integer, floats, strings and booleans. Input/output functions are accomplished with input() and print functions.

Task 1

In [1]:
```
name = "Jiya Jha"
age = 19

print(f"My name is {name} and I am {age} years old.")
```

```
My name is Jiya Jha and I am 19 years old.
```

Task 2

In [2]:
```
# Different data types
a = 10            # int
b = 3.14          # float
c = "Hello"       # string
d = True          # boolean

# Manipulation
print(a + 5)
print(b * 2)
print(c.upper())
print(not d)
```

```
15
6.28
HELLO
False
```

Part 2: Data Structures

Python has intrinsic data structures including lists, tuples, sets and dictionaries. Lists are flexible and are applied to an ordered data and dictionaries are a common store of pairs of key-value. The structures assist in the effective storage and handling of data.

Task 3

In [ ]:
```
students = ["Alice", "Bob", "Charlie"]

def manage_students():
    while True: # Keep the program running until the user wants to exit
        print(f"\nCurrent Students: {students}")
        action = input("Choose (add/remove/search/exit): ").lower()

        if action == "add":
            name = input("Enter name: ")
            students.append(name)
            print(f"{name} added.")

        elif action == "remove":
            name = input("Enter name to remove: ")
            if name in students:
                students.remove(name)
                print(f"{name} removed.")
            else:
                print("Name not found.")

        elif action == "search":
            name = input("Search name: ")
            print("Result: Found" if name in students else "Result: Not Found")

        elif action == "exit":
            print("Exiting program.")
            break # Breaks the while loop

        else:
            print("Invalid option, try again.")

manage_students()
```

```
Current Students: ['Alice', 'Bob', 'Charlie']
Invalid option, try again.

Current Students: ['Alice', 'Bob', 'Charlie']
Exiting program.
```

Task 4

In [ ]:
```
student_marks = {"Alice": 85, "Bob": 92}

def manage_marks():
    # Adding a new entry
    name = input("Enter student name: ")
    mark = float(input(f"Enter marks for {name}: "))
```

```
    student_marks[name] = mark

    # Computing average
    all_marks = student_marks.values()
    average = sum(all_marks) / len(all_marks)

    print(f"\nUpdated Marks: {student_marks}")
    print(f"The average score is: {average:.2f}")

manage_marks()
```

```
Updated Marks: {'Alice': 85, 'Bob': 2.0}
The average score is: 43.50
```

Part 3: Control Flow

Control flow enables decision making through use of the if-else statement and repetition through the use of for and while loop. These are necessary in application of logic like checking of prime numbers or creation of menu driven programmes.

Task 5

In [4]:
```
num = int(input("Enter number: "))

if num > 1:
    for i in range(2, num):
        if num % i == 0:
            print("Not Prime")
            break
    else:
        print("Prime")
else:
    print("Not Prime")
```

```
Prime
```

Task 6

In [17]:
```
while True:
    print("\n--- Arithmetic Menu ---")
    print("1. Add")
    print("2. Subtract")
    print("3. Exit")

    choice = input("Select: ")

    if choice == '3':
        print("Exiting...")
        break

    if choice == '1' or choice == '2':
        # Taking inputs
        a = float(input("Enter A: "))
        b = float(input("Enter B: "))
```

```
        if choice == '1':
            print(f"\n>>> THE RESULT IS: {a + b}")
        elif choice == '2':
            print(f"\n>>> THE RESULT IS: {a - b}")

        # This line keeps the result on the screen until you press Enter
        input("\nPress Enter to return to menu...")
    else:
        print("Please choose 1, 2, or 3.")
```

```
--- Arithmetic Menu ---
1. Add
2. Subtract
3. Exit

>>> THE RESULT IS: 12.0

--- Arithmetic Menu ---
1. Add
2. Subtract
3. Exit

>>> THE RESULT IS: 12.0

--- Arithmetic Menu ---
1. Add
2. Subtract
3. Exit
Exiting...
```

Part 4: Pandas

Pandas is employed in manipulating and analyzing data. It enables easy loading of datasets, cleaning of missing values, data filtering, data grouping, and statistical analysis easily using DataFrames.

Task 7

In [20]:

```
import pandas as pd

# Load the CSV file
df = pd.read_csv('nepal_earthquake_2015.csv')

# Data Cleaning: Removing redundant columns and filling missing values
df_cleaned = df.drop(columns=['Unnamed: 0'])
df_cleaned = df_cleaned.fillna("Unknown")

# Summary Statistics (shows count, unique values, top frequency, etc.)
print(df_cleaned.describe(include='all'))
```

```
             User_ID Username                 Tweet_Date  \
count   1.262000e+03     1262                       1262
unique           NaN      251                        667
top              NaN   NRaule  2015-04-25 20:48:57+00:00
freq             NaN       90                          6
mean    5.925416e+17      NaN                        NaN
std     8.323958e+14      NaN                        NaN
min     5.918475e+17      NaN                        NaN
25%     5.919321e+17      NaN                        NaN
50%     5.922332e+17      NaN                        NaN
75%     5.928085e+17      NaN                        NaN
max     5.952665e+17      NaN                        NaN

                                                  Tweet  \
count                                              1262
unique                                              684
top     This is The List Citizens Who Are Still in Nep...
freq                                                  7
mean                                                NaN
std                                                 NaN
min                                                 NaN
25%                                                 NaN
50%                                                 NaN
75%                                                 NaN
max                                                 NaN

                                            Tweet_Link
count                                             1262
unique                                             693
top     https://twitter.com/AbhiyanGhimire/status/5952...
freq                                                 2
mean                                               NaN
std                                                NaN
min                                                NaN
25%                                                NaN
50%                                                NaN
75%                                                NaN
max                                                NaN
```

Task 8

In [24]:

```python
import pandas as pd

# 1. Data Selection: Choosing specific columns
selection = df[['Username', 'Tweet']]
print("--- Selection (First 5 Rows) ---")
print(selection.head())

# 2. Data Filtering: Finding tweets that contain the word 'help'
filtered = df[df['Tweet'].str.contains('help', case=False)]
print("\n--- Filtering (Tweets containing 'help') ---")
print(filtered[['Username', 'Tweet']].head())

# 3. Grouping: Counting how many tweets each user posted
grouped = df.groupby('Username').size().sort_values(ascending=False)
print("\n--- Grouping (Top 5 Most Active Users) ---")
print(grouped.head())

# 4. Sorting: Sorting the data by Tweet Date
```

```python
sorted_df = df.sort_values(by='Tweet_Date')
print("\n--- Sorting (Oldest Tweets First) ---")
print(sorted_df[['Tweet_Date', 'Username']].head())

# 5. Data Transformation: Creating a new column for Tweet Length
df['Tweet_Length'] = df['Tweet'].apply(len)
print("\n--- Transformation (Added Tweet_Length Column) ---")
print(df[['Tweet', 'Tweet_Length']].head())
```

```
--- Selection (First 5 Rows) ---
         Username                                              Tweet
0  AbhiyanGhimire  God save victims of earthquake.. #PrayForNepal...
1          9Suraz  Lamjung : the unique village in asia"ghale gau...
2        IamAmood  "Humanity do not have boundaries" In Nepal to ...
3    Prince_sezza  Earthquake ruins the life of thousands of peop...
4         qidr_an  We met a group of Malaysians who experienced t...

--- Filtering (Tweets containing 'help') ---
          Username                                               Tweet
0   AbhiyanGhimire  God save victims of earthquake.. #PrayForNepal...
2         IamAmood  "Humanity do not have boundaries" In Nepal to ...
8         NirmuniM             Please help earthquake victim in Nepal
12       LokJoshi4  Indian publicity regardaing earthquake is very...
18  Darktoothboy07  Glad to see Nepalese Youth are united to help ...

--- Grouping (Top 5 Most Active Users) ---
Username
NRaule             90
navizeapp          86
HenkilBernardRZ    52
QuakesZone         43
Ekendra            34
dtype: int64

--- Sorting (Oldest Tweets First) ---
                     Tweet_Date      Username
572   2015-04-25 06:13:36+00:00  changyyorong
1233  2015-04-25 06:13:36+00:00  changyyorong
571   2015-04-25 06:17:18+00:00      paavan11
1232  2015-04-25 06:17:18+00:00      paavan11
1244  2015-04-25 06:19:08+00:00       muditkh

--- Transformation (Added Tweet_Length Column) ---
                                              Tweet  Tweet_Length
0  God save victims of earthquake.. #PrayForNepal...           116
1  Lamjung : the unique village in asia"ghale gau...           112
2  "Humanity do not have boundaries" In Nepal to ...           135
3  Earthquake ruins the life of thousands of peop...            67
4  We met a group of Malaysians who experienced t...           138
```

Part 5: NumPy

NumPy is applied in making numerical computations. It offers rapid array operations, matrix multiplication, reshaping and mathematical functions and is therefore critical to scientific computing.

Task 9

In [26]:
```
import numpy as np

# 1. Create Array: Converting a Python list into a high-performance NumPy array.
arr = np.array([1, 2, 3, 4, 5, 6])
print("1. Original Array:", arr)

# 2. Reshaping: Changing the shape of the data without changing the data itself.
# Converts 1D (6 elements) to 2D (2 rows, 3 columns)
matrix = arr.reshape(2, 3)
print("\n2. Reshaped Matrix (2x3):\n", matrix)

# 3. Slicing: Extracting a specific portion of the array.
# [1:5] means start at index 1 and go up to (but not including) index 5.
slice_arr = arr[1:5]
print("\n3. Sliced Array (Index 1 to 4):", slice_arr)

# 4. Dot Product: Sum of the products of corresponding elements.
# Calculated as: (1*3) + (2*4) = 3 + 8 = 11
v1 = np.array([1, 2])
v2 = np.array([3, 4])
dot = np.dot(v1, v2)
print("\n4. Dot Product of v1 and v2:", dot)

# 5. Matrix Multiplication: Following linear algebra rules for multiplying matri
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])
mat_mul = np.matmul(m1, m2)
print("\n5. Matrix Multiplication (m1 @ m2):\n", mat_mul)
```

```
1. Original Array: [1 2 3 4 5 6]

2. Reshaped Matrix (2x3):
 [[1 2 3]
 [4 5 6]]

3. Sliced Array (Index 1 to 4): [2 3 4 5]

4. Dot Product of v1 and v2: 11

5. Matrix Multiplication (m1 @ m2):
 [[19 22]
 [43 50]]
```

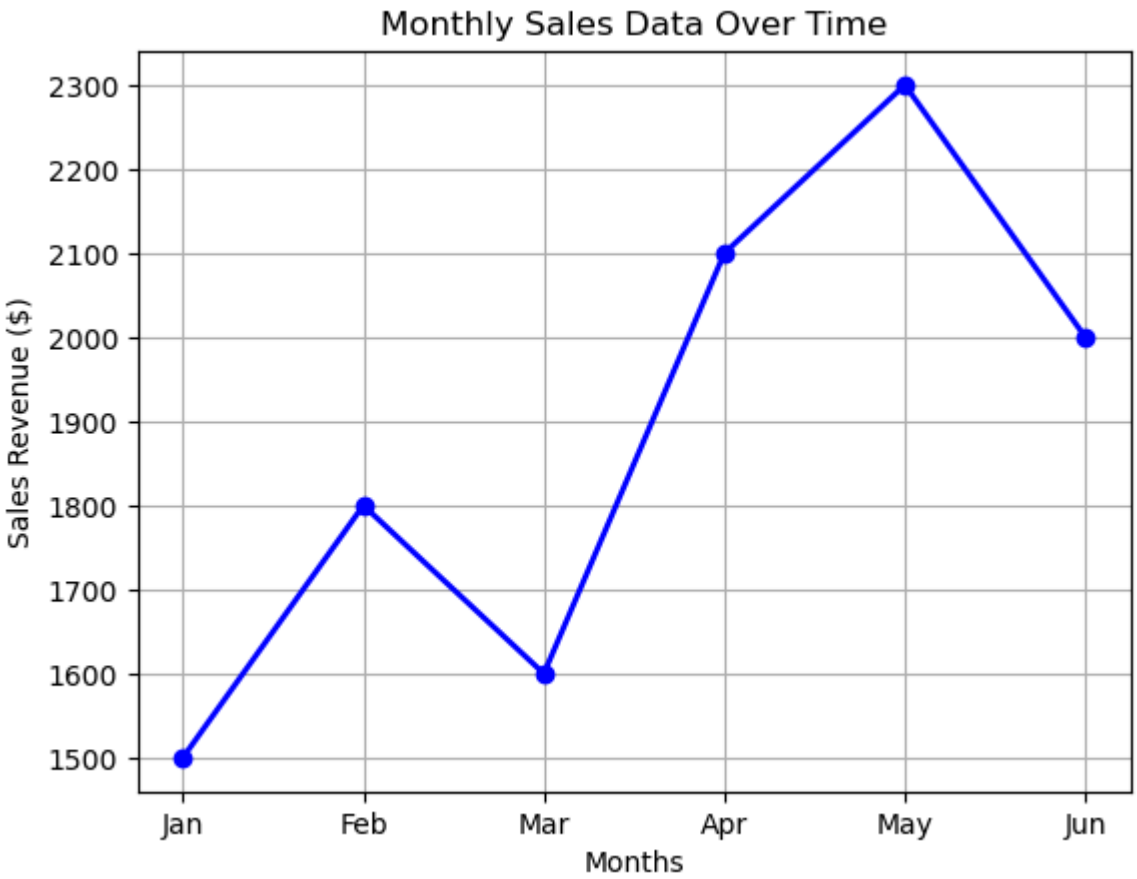
Part 6: Visualization

Trends and patterns are understood by the use of data visualization. The most popular libraries in making graphs are Matplotlib and seaborn where bar charts, graphs like line plots, and histograms are created.

Task 10

In [27]:
```
import matplotlib.pyplot as plt

# Data: Months and corresponding Sales figures
months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
```

```
sales = [1500, 1800, 1600, 2100, 2300, 2000]

# Creating the plot
plt.plot(months, sales, marker='o', color='blue', linestyle='-', linewidth=2)

# Adding Labels and Title
plt.title('Monthly Sales Data Over Time')
plt.xlabel('Months')
plt.ylabel('Sales Revenue ($)')

# Adding a grid for better readability
plt.grid(True)

# Display the result
plt.show()
```

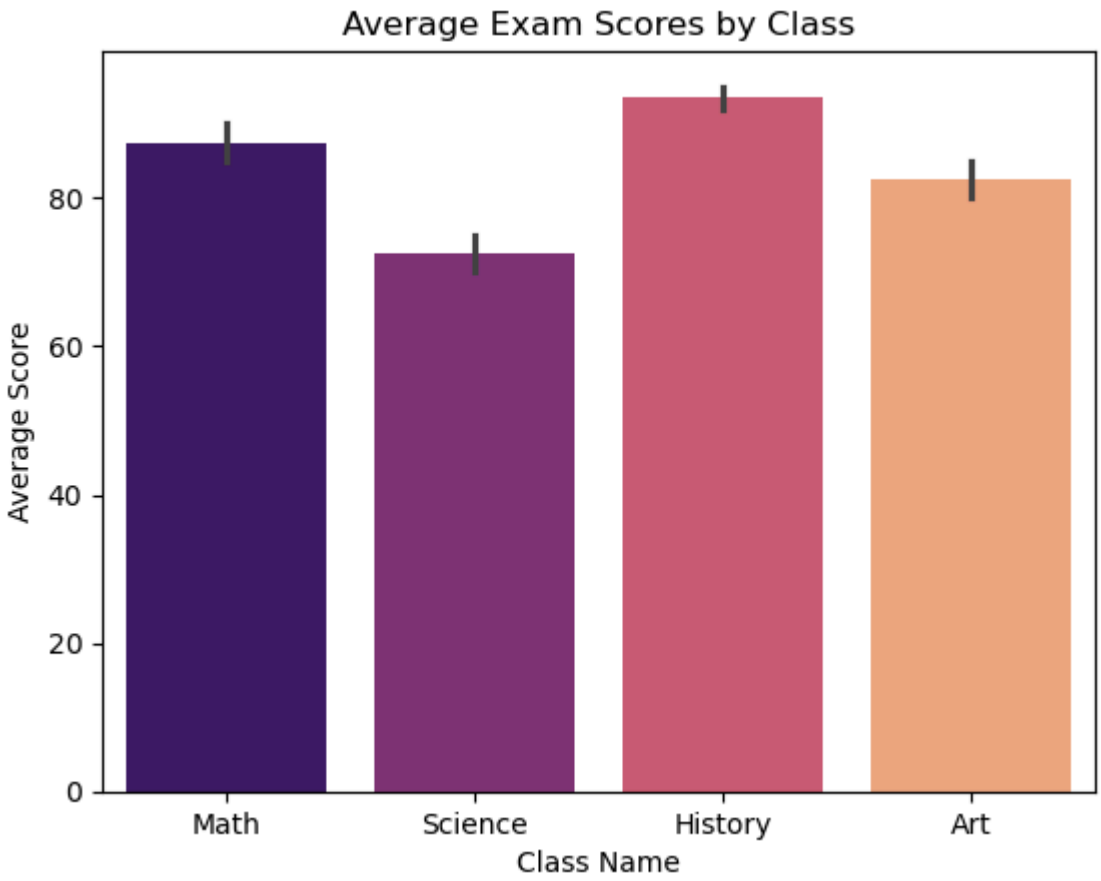


Task 11

In [28]:
```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# 1. Create a Dictionary with student data
data = {
    'Class': ['Math', 'Math', 'Science', 'Science', 'History', 'History', 'Art',
    'Score': [85, 90, 70, 75, 95, 92, 80, 85]
}

# 2. Convert to a Pandas DataFrame
df_students = pd.DataFrame(data)
```

```
# 3. Create the Bar Chart using Seaborn
# Seaborn automatically calculates the average (mean) for each class
sns.barplot(x='Class', y='Score', data=df_students, palette='magma')

# 4. Adding Title and Labels
plt.title('Average Exam Scores by Class')
plt.xlabel('Class Name')
plt.ylabel('Average Score')

# Display the result
plt.show()
```

```
C:\Users\jhaji\AppData\Local\Temp\ipykernel_32680\3817217920.py:16: FutureWarnin
g:

Passing `palette` without assigning `hue` is deprecated and will be removed in v
0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effe
ct.

  sns.barplot(x='Class', y='Score', data=df_students, palette='magma')
```



Part 7: Machine Learning

Scikit-learn has been utilized to create machine learning models. It is focusing on the regression, classification and clustering algorithms. The data is trained on models and passed on to make forecasts.

Task 12

In [29]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans
from sklearn.datasets import make_regression, load_iris
from sklearn.metrics import mean_squared_error, accuracy_score

# 1. Regression (Predicting a continuous value)
X_reg, y_reg = make_regression(n_samples=100, n_features=1, noise=15, random_sta
X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_reg, y_reg, test_s

reg_model = LinearRegression().fit(X_train_r, y_train_r)
prediction = reg_model.predict([[1.5]]) # Predicting for a new value 1.5
print(f"Regression Prediction for 1.5: {prediction[0]}")

# 2. Classification (GNB, Decision Tree, Random Forest)
iris = load_iris()
X_c, y_c = iris.data, iris.target
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(X_c, y_c, test_size=

# Initializing models
models = {
    "Gaussian NB": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier()
}

# Training and testing each model
for name, model in models.items():
    model.fit(X_train_c, y_train_c)
    acc = accuracy_score(y_test_c, model.predict(X_test_c))
    print(f"{name} Accuracy: {acc:.2%}")

# 3. Clustering (K-Means)
# Unsupervised learning to group data into 3 clusters
kmeans = KMeans(n_clusters=3, n_init=10).fit(X_c)
print(f"First 5 Cluster Labels: {kmeans.labels_[:5]}")
```

```
Regression Prediction for 1.5: 73.42880111926995
Gaussian NB Accuracy: 91.11%
Decision Tree Accuracy: 86.67%
Random Forest Accuracy: 93.33%
First 5 Cluster Labels: [0 0 0 0 0]
```

```
c:\Users\jhaji\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1429: UserW
arning: KMeans is known to have a memory leak on Windows with MKL, when there are
less chunks than available threads. You can avoid it by setting the environment v
ariable OMP_NUM_THREADS=1.
  warnings.warn(
```

Part 8: Optimization & Debugging

Optimization enhances efficiency of code through such techniques as list comprehensions. The process of debugging assists in discovering and correcting bugs with the help of print statements and exception handling.

Task 13

In [30]:
```
def unoptimized_squares(n):
    result = []
    for i in range(n):
        result.append(i**2) # append() is slow inside a loop
    return result
```

In [31]:
```
def optimized_squares(n):
    # List comprehension is faster and more readable
    return [i**2 for i in range(n)]

# Example usage
print(optimized_squares(10))
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Task 14

In [ ]:
```
def calculate_average(numbers): # FIX 1: Added the missing colon (:)
    total_sum = 0 # Pro-tip: Avoid using 'sum' as a variable name because it's a

    for i in range(len(numbers)):
        # FIX 2: Changed [i+1] to [i]
        # numbers[i+1] would try to access an index that doesn't exist at the en
        total_sum = total_sum + numbers[i]

    return total_sum / len(numbers)

# Testing the corrected function
result = calculate_average([10, 20, 30])
print(f"The average is: {result}")
```

```
The average is: 20.0
```

Explanation of Debugging Techniques Used:

Syntax Correction: Added the : at the end of the def line. In Python, blocks are initiated with colons.

Dry-Running (Logical Fix): I realized that if a list has 3 items, the indices are 0, 1, 2. Using i+1 inside the loop would eventually try to access index 3, causing an IndexError. I changed it to numbers[i].

Traceback Analysis: Interpreted the SyntaxError: expected ':' provided by the Python interpreter to locate the first crash point.

Variable Renaming: Changed sum to total_val to ensure the built-in Python sum() function remains accessible.